

WEB APPLICATION SECURITY ASSESSMENT REPORT

Target Applications: DVWA (Damn Vulnerable Web Application) & OWASP Juice Shop

Assessment Type: Grey-Box Web Application Penetration Test

Classification: Internal Use Only / Academic Security Assessment

1. EXECUTIVE SUMMARY

A comprehensive web application security assessment was conducted on DVWA (Damn Vulnerable Web Application) and OWASP Juice Shop to identify, evaluate, and document security vulnerabilities. The assessment combined automated scanning tools (Nmap, Nikto, OWASP ZAP) with manual verification and exploitation techniques following the OWASP Testing Guide (OTGv4) methodology.

The assessment identified multiple Critical and High-severity vulnerabilities that allow authentication bypass, unauthorized administrative access, cookie theft, session hijacking, and direct object manipulation.

Vulnerability Count Summary

- **Critical Vulnerabilities:** 4 (SQL Injection, Broken Authentication, Insecure Password Hashing, Session Management Flaws)
- **High Vulnerabilities:** 6 (Reflected XSS, Stored XSS, IDOR, SSRF, Missing Rate Limiting, Sensitive Data Exposure)
- **Medium Vulnerabilities:** 6 (Missing Security Headers, Software Version Disclosure, CSRF Risks)

2. INFORMATION GATHERING & RECONNAISSANCE

2.1 Port Scanning & Service Discovery (Nmap)

Command:

```
nmap -sV -sC -p- 127.0.0.1 -oN nmap_recon.txt
```

Port	State	Service
22/tcp	Open	SSH
80/tcp	Open	HTTP
3000/tcp	Open	HTTP
3306/tcp	Open	MySQL

2.2 Web Server Analysis & Fingerprinting (Nikto)

Command:

```
nikto -h http://localhost:80
nikto -h http://localhost:3000
```

Key Scan Findings:

- Server Header Disclosure:** Apache/2.4.6 (CentOS) and Express (X-Powered-By: Express) signatures publicly exposed.
- Missing Anti-Clickjacking Header:** X-Frame-Options header missing across multiple endpoints.
- Missing MIME Sniffing Protection:** X-Content-Type-Options header not configured.
- Missing Content Security Policy:** Absence of Content-Security-Policy (CSP) header allows inline script execution.

3. AUTOMATED VULNERABILITY SCANNING RESULTS (OWASP ZAP)

Vulnerability Class	Count	Risk Level
SQL Injection	4	CRITICAL
Cross-Site Scripting	6	HIGH

Vulnerability Class	Count					Risk Level
(Reflected)						
Weak / Broken Authentication	3					CRITICAL
Insecure Direct Object Reference (IDOR)	2					HIGH
Missing Security Headers	5					MEDIUM
	Server-Side Request Forgery	2	HIGH	Remote URL Import	Unvalidated URL fetching allows internal network scanning.	

4. DETAILED VULNERABILITY FINDINGS & CODE REMEDIATIONS

4.1 SQL Injection (SQLi) - Authentication Bypass

- **Vulnerability ID:** SEC-001
- **Risk Level:** CRITICAL (CVSS v3.1: 9.8)
- **OWASP Top 10:** A03:2021 – Injection
- **Affected Endpoint:** `/login.php` (DVWA) / `/rest/user/login` (Juice Shop)

Description

The login form concatenates user input directly into SQL strings without sanitization or parameterization, allowing malicious SQL logic execution.

Steps to Reproduce

1. Navigate to the login page.
2. Enter the following payload into the Username field: `admin' --` or `admin' #`
3. Enter any dummy value in the Password field.
4. Click Login.
5. **Result:** Authenticated successfully as administrator without a valid password.

Code Remediation Snippets

PHP (Using PDO Prepared Statements)

//  VULNERABLE: Direct string concatenation

```
$username = $_POST['username'];
```

```
$query = "SELECT * FROM users WHERE user = '$username' AND password = '$password'";
```

```
$result = mysqli_query($conn, $query);
```

//  SECURE: Prepared Statement with Parameter Binding

```

$stmt = $pdo->prepare('SELECT id, username, password FROM users WHERE
username = :username');

$stmt->execute(['username' => $_POST['username']]);

$user = $stmt->fetch();

if ($user && password_verify($_POST['password'], $user['password'])) {

    // Authenticated successfully

}

```

Node.js (Using Parameterized Query Array)

```

// 🚫 VULNERABLE: String interpolation

const query = `SELECT * FROM users WHERE email = '${req.body.email}'`;

// ✅ SECURE: Parameterized SQL query

const text = 'SELECT id, password_hash, role FROM users WHERE email = $1';

const values = [req.body.email];

const result = await db.query(text, values);

```

4.2 Cross-Site Scripting (XSS) - Reflected & Stored

- **Vulnerability ID:** SEC-002
- **Risk Level:** HIGH (CVSS v3.1: 8.2)
- **OWASP Top 10:** A03:2021 – Injection
- **Affected Endpoints:** [/vulnerabilities/xss_r/](#) (DVWA) / Search Bar & Comment Section

Description

User-controlled parameters are reflected back into the HTTP response without HTML entity encoding, allowing arbitrary JavaScript execution in the user's browser.

Steps to Reproduce

1. Access the comment or search input field.
2. Submit the following payload: `<script>alert('XSS Vulnerability Found')</script>` or `<script>alert(document.cookie)</script>`
3. Result: An alert pop-up executes displaying session details or document cookies.

Code Remediation Snippets

PHP (HTML Entity Encoding)

p

```
// 🚫 VULNERABLE: Direct output rendering
echo "Welcome, " . $_GET['name'] . "";
```

```
// ✅ SECURE: Context-aware HTML entity encoding
$cleanName = htmlspecialchars($_GET['name'], ENT_QUOTES |
ENT_SUBSTITUTE, 'UTF-8');
echo "Welcome, " . $cleanName . "";
```

Node.js / Express (Input Sanitization)

```
``javascript
```

```
// 🚫 VULNERABLE: Rendering unescaped string
res.send(`<h1>Results for: ${req.query.q}</h1>`);
```

```
// ✅ SECURE: Sanitizing input using sanitize-html

const sanitizeHtml = require('sanitize-html');

const cleanQuery = sanitizeHtml(req.query.q, {

  allowedTags: [],

  allowedAttributes: {}

});
```

```
res.send('<h1>Results for: ${cleanQuery}</h1>');
```

4.3 Broken Authentication & Session Management

- **Vulnerability ID:** SEC-003
- **Risk Level:** **CRITICAL** (CVSS v3.1: 9.1)
- **OWASP Top 10:** A07:2021 – Identification and Authentication Failures
- **Affected Endpoints:** Authentication portal & Session Token generation

Description

The application permits default/weak credentials (`admin/password`, `user/user`), lacks account lockout policies against brute-force attacks, and sets session cookies without security flags (`HttpOnly`, `Secure`, `SameSite`).

Steps to Reproduce

1. Capture a login request using Burp Suite Proxy.
2. Send request to Burp Intruder and load a credential wordlist.
3. Perform brute-force testing; notice lack of rate limiting, CAPTCHA, or IP throttling.

Code Remediation Snippets

PHP Session & Password Security

```
//  VULNERABLE: Weak MD5 hash & flagless cookie
```

```
$hashedPassword = md5($_POST['password']);
```

```
setcookie("session_id", session_id());
```

```
//  SECURE: Argon2/Bcrypt & Secure HTTP-only Cookie Configuration
```

```
$secureHash = password_hash($_POST['password'],  
PASSWORD_ARGON2ID);
```

```
if (password_verify($_POST['password'], $user['password_hash'])) {
```

```
    session_regenerate_id(true); // Prevent Session Fixation
```

```
setcookie("session_token", $token, [  
  
  'expires' => time() + 3600,  
  
  'path'    => '/',  
  
  'domain'  => 'example.com',  
  
  'secure'  => true,    // Transmit over HTTPS only  
  
  'httponly' => true,   // Prevent JavaScript cookie theft (XSS)  
  
  'samesite' => 'Strict' // CSRF protection  
  
]);  
  
}
```

Node.js / Express Session Security

```
const bcrypt = require('bcrypt');  
  
const session = require('express-session');  
  
// 🛡️ SECURE: Express Session Cookie Configuration  
  
app.use(session({  
  
  name: '__Host-sess',  
  
  secret: process.env.SESSION_SECRET,  
  
  resave: false,  
  
  saveUninitialized: false,  
  
  cookie: {  
  
    httpOnly: true,    // Block client-side JS access
```

```
secure: true,    // Enforce HTTPS

sameSite: 'strict', // Strict CSRF protection

maxAge: 3600000    // 1 hour expiration

}

});
```

4.4 Insecure Direct Object References (IDOR)

- **Vulnerability ID:** SEC-004
- **Risk Level:** HIGH (CVSS v3.1: 7.5)
- **OWASP Top 10:** A01:2021 – Broken Access Control
- **Affected Endpoint:** </vulnerabilities/view-source/?id=1> / User Profile APIs

Description

The backend uses sequential integer keys ([id=1](#), [id=2](#)) without verifying whether the requesting user owns the object. An attacker can manipulate parameters to access unauthorized records.

Steps to Reproduce

1. Log in as a low-privileged user.
2. Intercept profile viewing URL:
<http://localhost/vulnerabilities/view-source/?id=1>
3. Modify [id](#) parameter from [1](#) to [2](#) or [3](#).
4. **Result:** Successfully accessed profile data belonging to other users without authorization checks.

Remediation Guidelines

1. Enforce backend authorization checks verifying ownership before returning resources.
2. Replace sequential integer IDs with cryptographically secure UUIDs (v4).

4.5 Server-Side Request Forgery (SSRF)

- **Vulnerability ID:** SEC-005

- **Risk Level: HIGH** (CVSS v3.1: 8.6)
- **OWASP Top 10:** A10:2021 – Server-Side Request Forgery
- **Affected Endpoints:** [/vulnerabilities/ssrf/](#) (DVWA) / [/rest/products/reviews](#) (Juice Shop)

Description

The application fetches remote resources based on user-supplied URLs without proper validation or IP filtering. An attacker can force the server to issue requests to internal services or cloud metadata endpoints.

Steps to Reproduce

1. Access the image URL import or remote resource fetch interface.
2. Intercept the request and modify the target URL parameter to internal targets (e.g., <http://169.254.169.254/latest/meta-data/>).
3. Submit the request and observe server responses disclosing internal service details.

Code Remediation Snippets

PHP (URL Whitelisting)

```
// ⓘ SECURE: Whitelist allowed domains and enforce HTTPS
$url = $_GET['url'];
$allowed_domains = ['trusted-assets.com', 'images.provider.net'];
$parsed_url = parse_url($url);
if (in_array($parsed_url['host'], $allowed_domains) &&
    $parsed_url['scheme'] === 'https') {
    echo file_get_contents($url);
}
```

Node.js (DNS Lookup & Private IP Check)

```
const ip = require('ip');
const dns = require('dns').promises;
// ⓘ SECURE: Check if resolved IP is private before fetching
const address = await dns.resolve4(new URL(req.body.url).hostname);
if (!ip.isPrivate(address[0])) {
    const response = await axios.get(req.body.url);
}
```

5. VULNERABILITY SUMMARY & RISK MATRIX

Vulnerability Title	Severity	CVSS v3.1	
SQL Injection	CRITICAL	9.8	
Broken Authentication	CRITICAL	9.1	
Reflected / Stored XSS	HIGH	8.2	
Insecure Direct Object References (IDOR)	HIGH	7.5	
Missing Security Headers	MEDIUM	6.1	

6. ACTIONABLE REMEDIATION ROADMAP

Priority 1: Immediate Actions (Critical Risk)

- Replace all dynamic SQL string operations with parameterized queries / prepared statements across PHP and Node.js codebases.
- Change default installation passwords (**admin/password**) immediately.
- Enforce account lockout policies (5 failed attempts) and rate limiting on authentication endpoints.

Priority 2: Short-Term Actions (High Risk)

1. Implement context-aware HTML entity encoding on all user output parameters.
2. Enforce backend access control checks (RBAC) to validate user authorization for resource IDs.
3. Apply `HttpOnly`, `Secure`, and `SameSite=Strict` flags to all session cookies.

Priority 3: Medium-Term Actions (Medium Risk)

1. Configure missing HTTP Security Headers:
 - `Content-Security-Policy: default-src 'self'`
 - `X-Frame-Options: DENY`
 - `X-Content-Type-Options: nosniff`
 - `Strict-Transport-Security: max-age=31536000; includeSubDomains`
2. Disable verbose error messages and server header signatures in production environments.

7. CONCLUSION

The security assessment identified multiple critical and high-severity vulnerabilities across the target applications. Unrestricted SQL Injection and Broken Authentication mechanisms represent immediate threats that could lead to full system compromise, data exfiltration, and account takeover.

By implementing the parameterized query implementations, context-aware encoding, secure cookie parameters, and access controls detailed in Section 4, the organization can effectively remediate these vulnerabilities and achieve a robust security posture.